CI/CD Interview Preparation Guide

Structured Answers for DevOps / Cloud / Platform Engineer Roles

1. CI/CD Fundamentals

Q: What is CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery/Deployment.
- It is a practice that automates the steps from writing code to deploying it to production.
- Goal: Deliver software faster, more reliably, and with fewer manual steps.

Q: What is Continuous Integration (CI)?

- Developers push code changes frequently (multiple times per day) to a shared repository.
- Every push automatically triggers a build and automated tests.
- Benefit: Catches bugs early before they grow into larger problems.

Q: What is Continuous Delivery?

- The code is always in a deployable state after passing all automated tests.
- Deployment to production requires a manual approval or trigger.
- Think of it as: 'The software is ready to ship anytime — a human decides when.'

Q: What is Continuous Deployment?

- Every change that passes all automated tests is automatically deployed to production.
- No manual approval step — fully automated end to end.
- Used by companies that release many times per day (e.g., Netflix, Amazon).

Q: Difference between Continuous Delivery and Continuous Deployment?

- Continuous Delivery: Code is ready to deploy, but a human approves the final push.
- Continuous Deployment: No human approval — code goes live automatically if tests pass.
- Key difference: Manual gate (Delivery) vs Fully automated (Deployment).

Q: Why is CI/CD important?

- Speeds up the software release cycle.
- Reduces integration bugs by merging small, frequent changes.
- Improves code quality through automated testing and scanning.
- Enables faster feedback to developers.
- Reduces risk of large, infrequent releases.

Q: What are the typical stages of a CI/CD pipeline?

- Source: Code is pushed to GitHub/GitLab triggering the pipeline.
- Build: Application is compiled or packaged (e.g., Maven build, Docker image build).
- Test: Automated unit, integration tests are run.
- Security Scan: Code scanned with SonarQube; Docker image scanned with Trivy.
- Artifact Storage: Built image is pushed to ECR or Docker Hub.
- Deploy: Application is deployed to staging, then production (EKS, EC2, etc.).
- Notify: Team receives Slack or email notification on success or failure.

2. Git & Source Control

Q: Why is Git important in CI/CD?

- Git is the source of truth for all code changes.
- CI/CD pipelines are triggered by Git events (push, pull request, merge).
- Branching strategies control what gets deployed where.

Q: What branching strategies have you used?

- GitFlow: Uses feature, develop, release, hotfix, and main branches. Suitable for scheduled releases.
- Trunk-Based Development: Developers push small changes directly to main/trunk frequently. Suits CI/CD well.
- Feature Branch Workflow: Each feature gets its own branch; merged via pull request after review.

Q: What is a Pull Request (PR)?

- A request to merge code from one branch into another (usually into main).
- It enables code review, automated checks, and discussion before merging.
- In CI/CD: PRs often trigger automated builds and tests automatically.

Q: How do webhooks work in CI/CD?

- A webhook is an HTTP callback that GitHub sends to Jenkins/GitHub Actions when an event occurs.
- Example: When a developer pushes code, GitHub sends a POST request to Jenkins, which triggers the pipeline.
- Webhooks replace manual polling, making pipelines fast and event-driven.

Q: How do you handle merge conflicts?

- Pull the latest changes from the base branch frequently to stay updated.
- Use small, frequent commits to reduce the chance of conflicts.
- Resolve conflicts locally using git merge or git rebase, then push.
- In CI pipelines, a merge conflict will cause the build to fail — developer must resolve it manually.

3. Jenkins Fundamentals

Q: What is Jenkins and why use it?

- Jenkins is an open-source automation server used to build CI/CD pipelines.
- It supports hundreds of plugins for Git, Docker, Kubernetes, AWS, Slack, and more.
- It is self-hosted, giving full control over the environment.
- Widely used in enterprises for its flexibility and large community.

Q: What is a Jenkins Pipeline?

- A pipeline is a set of automated steps defined in code (Jenkinsfile) that runs the CI/CD process.
- It defines stages like Build, Test, Deploy.
- Stored as code in the repository, making it version-controlled.

Q: Freestyle Job vs Pipeline Job?

- Freestyle Job: Simple, UI-configured job. No code required. Limited flexibility.
- Pipeline Job: Defined in a Jenkinsfile using Groovy DSL. Supports complex workflows, parallelism, conditionals.
- Best practice: Always use Pipeline jobs for real-world CI/CD.

Q: What is a Jenkinsfile?

- A text file written in Groovy DSL that defines the entire pipeline as code.
- Stored in the root of the Git repository.
- Enables pipeline versioning, code reviews, and reusability.

Q: Explain Jenkins Architecture (Master-Agent).

• Master (Controller): Manages jobs, schedules builds, stores configurations, and serves the UI.
• Agent (Node): Executes the actual build steps. Can be on separate machines.
• Why agents? Distribute workload, run jobs in parallel, use environment-specific machines (Linux, Windows).
• Connection: Agents connect to master via SSH or JNLP protocol.

4. Jenkins Pipeline Deep Dive

Q: Declarative vs Scripted Pipeline?

• Declarative: Structured, uses predefined syntax with 'pipeline {}' block. Easier to read and maintain.
• Scripted: Full Groovy code starting with 'node {}'. More flexible but complex.
• Recommendation: Use Declarative for most pipelines; use Scripted only when advanced logic is needed.

Q: Explain the following Jenkinsfile:

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'mvn package' } }
    stage('Test') { steps { sh 'mvn test' } }
  }
}
```

• pipeline {}: Declares this as a Declarative Pipeline.
• agent any: Run this pipeline on any available Jenkins agent.
• stages {}: Container for all pipeline stages.
• stage('Build'): Defines a named stage. Visible in Jenkins UI.
• sh 'mvn package': Runs a shell command to build the Java app using Maven.
• stage('Test'): Runs Maven tests after the build succeeds.

Q: What is the post block in Jenkins?

• Runs steps after pipeline execution based on the result.
• always: Runs no matter the result (e.g., cleanup, notifications).
• success: Runs only when pipeline succeeds.
• failure: Runs only when pipeline fails (e.g., alert the team).

Q: How do you store credentials in Jenkins?

• Use Jenkins Credentials Store (Manage Jenkins > Credentials).
• In the pipeline, reference with: credentials('credential-id').
• Types: Username/password, Secret text, SSH key, AWS credentials.
• Never hardcode secrets in Jenkinsfile.

Q: How do you run stages in parallel in Jenkins?

• Use the parallel keyword inside a stage to run multiple sub-stages simultaneously.
• Example: Run unit tests and security scans at the same time to save time.
• Parallel execution reduces total pipeline duration significantly.

5. GitHub Actions

Q: What is GitHub Actions?

• A CI/CD platform built directly into GitHub.
• Workflows are defined in YAML files stored in .github/workflows/.
• No separate server needed — GitHub hosts and runs the workflows.

Q: Explain this GitHub Actions workflow:

```
name: Build
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

- name: Build: The display name of this workflow.
- on: push: This workflow triggers on every git push.
- jobs: build: Defines a job named 'build'.
- runs-on: ubuntu-latest: Use GitHub's Ubuntu runner to execute the job.
- uses: actions/checkout@v3: Pre-built action to check out (clone) the repository code.

Q: Self-hosted vs GitHub-hosted runners?

- GitHub-hosted: Managed by GitHub. Spins up fresh VMs (ubuntu-latest, windows-latest). No maintenance required.
- Self-hosted: Your own machine or server. Needed for private network access, special hardware, or cost savings at scale.

Q: How do secrets work in GitHub Actions?

- Stored in GitHub Settings > Secrets and Variables > Actions.
- Referenced in workflows as: ${{ secrets.MY_SECRET }}.
- Never printed in logs. GitHub masks them automatically.
- Common use: AWS credentials, Docker Hub tokens, kubeconfig.

Q: How does GitHub Actions differ from Jenkins?

- GitHub Actions: Cloud-native, no server to manage, tight GitHub integration, YAML-based.
- Jenkins: Self-hosted, more control, richer plugin ecosystem, Groovy-based.
- GitHub Actions is faster to set up; Jenkins suits complex enterprise environments.

6. Docker in CI/CD

Q: Why use Docker in CI/CD?

- Packages the application with all its dependencies into a portable image.
- Eliminates 'works on my machine' problems.
- Ensures the same environment in development, staging, and production.
- Docker images are versioned artifacts that can be promoted across environments.

Q: How do you build and push Docker images in a pipeline?

```
docker build -t my-app:v1.0 .
docker tag my-app:v1.0 123456.dkr.ecr.us-east-1.amazonaws.com/my-app:v1.0
docker push 123456.dkr.ecr.us-east-1.amazonaws.com/my-app:v1.0
```

- Build: Creates a Docker image from the Dockerfile.
- Tag: Labels the image with a version or commit SHA.
- Push: Uploads the image to ECR (AWS) or Docker Hub.

Q: Why avoid the 'latest' tag in production?

- 'latest' is ambiguous — you cannot tell which version is deployed.
- Rolling back is impossible if all versions are tagged 'latest'.
- Best practice: Use Git commit SHA or semantic versioning (e.g., v1.2.3) as image tags.

Q: What is a multi-stage Docker build?

- Uses multiple FROM statements in a single Dockerfile.
- Stage 1 (Build stage): Compile the application using a heavy image (e.g., maven:3.8).
- Stage 2 (Runtime stage): Copy only the compiled artifact into a lightweight image (e.g., openjdk:17-slim).
- Result: Smaller, more secure production images without build tools.

Q: How do you scan Docker images in a pipeline?

- Use Trivy: trivy image my-app:v1.0
- Trivy scans OS packages and application libraries for known CVEs.
- Fail the pipeline if CRITICAL vulnerabilities are found.
- Best practice: Scan after image build, before pushing to registry.

7. Kubernetes Deployments via CI/CD

Q: How do pipelines deploy to Kubernetes / EKS?

- Step 1: Authenticate to EKS using AWS CLI: aws eks update-kubeconfig --name cluster-name
- Step 2: Apply Kubernetes manifests: kubectl apply -f deployment.yaml
- Step 3: Verify rollout: kubectl rollout status deployment/my-app
- IAM Role (IRSA or EC2 instance role) grants the Jenkins agent/runner permission to EKS.

Q: What are common Kubernetes deployment strategies?

- Rolling Update (Default): Replaces old pods with new ones gradually. Zero downtime. Easy to configure.
- Blue-Green: Two environments (blue=live, green=new). Switch traffic all at once. Instant rollback by switching back.
- Canary: Route a small % of traffic to new version first. Monitor, then gradually increase. Reduces blast radius.
- Recreate: Kill all old pods, then deploy new ones. Causes downtime. Use only for non-critical apps.

Q: How do you rollback a Kubernetes deployment?

- Quick rollback: kubectl rollout undo deployment/my-app
- Rollback to specific version: kubectl rollout undo deployment/my-app --to-revision=2
- In CI/CD: Tag images with commit SHA so you can redeploy a previous image at any time.

Q: How do you verify deployment success in a pipeline?

- kubectl rollout status deployment/my-app — waits until all pods are healthy or returns an error.
- kubectl get pods — check all pods are in Running state.
- Run a health check against the service endpoint (HTTP 200 response).

8. Terraform in CI/CD

Q: Why automate Terraform in CI/CD?

- Prevents manual infrastructure changes (drift from desired state).
- Enables Infrastructure as Code reviews via pull requests.
- Ensures consistent environments across dev, staging, and production.
- Provides audit trail of all infrastructure changes.

Q: What are the typical Terraform stages in a pipeline?

- terraform init: Initialize the working directory, download providers.
- terraform validate: Check syntax of Terraform code.
- terraform plan: Show what changes will be made — review before applying.
- Manual Approval: Human reviews the plan output before proceeding.

- terraform apply -auto-approve: Apply the approved plan to provision infrastructure.

Q: Why run terraform plan before apply?

- Plan shows exactly what resources will be created, modified, or destroyed.
- Prevents accidental deletion of critical resources (e.g., databases).
- Required for peer review in production environments.

Q: How do you store Terraform state in CI/CD?

- Use remote backend — typically AWS S3 with DynamoDB for state locking.
- Prevents two pipelines from modifying infrastructure simultaneously.
- Never commit .tfstate files to Git — they can contain sensitive values.

Q: How do you prevent accidental destroy in production?

- Use terraform plan output review with manual approval gate.
- Use lifecycle { prevent_destroy = true } in Terraform for critical resources.
- Separate state files for each environment (dev/staging/prod).
- Restrict production apply to specific branches (e.g., only main).

9. Security in CI/CD (SonarQube & Trivy)

Q: What is Shift Left Security?

- Moving security checks earlier in the development cycle.
- Instead of testing security at the end, scan code and images during the pipeline.
- Benefit: Cheaper and faster to fix vulnerabilities early than after deployment.

Q: What is SonarQube and what does it scan?

- SonarQube is a static code analysis tool.
- It scans source code for: bugs, code smells, security vulnerabilities, duplicated code.
- Quality Gates: Define rules (e.g., 'no new critical bugs'). Pipeline fails if gate is not passed.
- Supports Java, Python, JavaScript, Go, and many more languages.

Q: What is Trivy and what does it scan?

- Trivy is a fast, open-source vulnerability scanner by Aqua Security.
- Scans: Docker images, filesystems, Git repos, Kubernetes manifests.
- Detects vulnerabilities in: OS packages (apt, yum) and application libraries (npm, pip, Maven).
- Command: trivy image --severity HIGH,CRITICAL my-app:v1.0

Q: How do you manage secrets in pipelines?

- Jenkins: Use Jenkins Credentials Manager. Never hardcode in Jenkinsfile.
- GitHub Actions: Use GitHub Secrets. Reference via ${{ secrets.NAME }}.
- AWS: Use IAM Roles instead of access keys where possible.
- Advanced: Use HashiCorp Vault or AWS Secrets Manager for dynamic secret injection.

Q: How do you fail a build based on vulnerabilities?

- SonarQube: Configure a Quality Gate with thresholds. Pipeline uses sonar-scanner; fails if gate fails.
- Trivy: Use --exit-code 1 flag: trivy image --exit-code 1 --severity CRITICAL my-app:v1.0
- Result: Pipeline stops and alerts the team when vulnerabilities are found.

10. Deployment Strategies (Rolling, Blue-Green, Canary)

Q: What is Rolling Deployment?

- Kubernetes default strategy.

- Gradually replaces old pods with new ones — a few at a time.
- Zero downtime. If new pods fail health checks, rollout pauses automatically.
- Best for: Most applications. Simple and safe.

Q: What is Blue-Green Deployment?

- Maintain two identical environments: Blue (current live) and Green (new version).
- Deploy to Green first, run tests, then switch the load balancer to Green.
- Instant rollback: Just switch traffic back to Blue if anything goes wrong.
- Downside: Double the infrastructure cost.

Q: What is Canary Deployment?

- Deploy the new version to a small subset of users first (e.g., 5–10%).
- Monitor error rates and performance metrics.
- Gradually increase traffic to the new version if it's healthy.
- Best for: High-traffic apps where you want to test with real users before full rollout.

Q: When would you use each strategy?

- Rolling: Default choice. Low risk, zero downtime, cost-efficient.
- Blue-Green: Major releases, database migrations, when you need instant rollback.
- Canary: Testing new features with a subset of users, gradual rollouts.
- Recreate: Dev/test environments, or when a new version is completely incompatible with the old.

11. Monitoring, Notifications & Troubleshooting

Q: How do you monitor CI/CD pipelines?

- Jenkins: Built-in dashboard shows build history, stage timings, and logs.
- GitHub Actions: Workflow runs visible in the Actions tab with logs per step.
- Metrics to track: Build success rate, average build duration, time-to-deploy.

Q: How do you send notifications from pipelines?

- Jenkins: Use Slack Notification plugin or Email Extension plugin in the post block.
- GitHub Actions: Use actions/slack-notify or similar marketplace actions.
- Notify on: pipeline failure, successful deployment, quality gate breach.

Q: Troubleshooting approach for common failures:

- Jenkins build fails: Check console logs for exact error. Verify build dependencies and credentials.
- Docker build fails: Check Dockerfile syntax, base image availability, network access to package repos.
- Kubernetes deployment fails: kubectl describe pod and kubectl logs for details.
- SonarQube gate fails: Review issues in SonarQube dashboard; fix or mark as false positives.
- Trivy critical CVEs: Update base image, upgrade vulnerable dependencies, or apply suppression with justification.
- AWS credentials not working: Verify IAM role permissions, check if token has expired, confirm region is correct.
- Terraform apply fails: Read error message carefully; check for resource conflicts or state lock issues.

12. Real-World Scenario Answers

Q: Scenario: Developer pushes code and Jenkins fails during build. How do you troubleshoot?

- Step 1: Open Jenkins Console Output for the failed build.
- Step 2: Identify the exact error message and failing stage.

- Step 3: Check if it's a code issue (compilation error) or environment issue (missing dependency).
- Step 4: Reproduce locally if possible.
- Step 5: Fix the issue, push again, and verify pipeline passes.

Q: Scenario: Docker image builds but Kubernetes deployment fails. What do you check?

- kubectl get pods — check pod status (CrashLoopBackOff, ImagePullBackOff, etc.).
- kubectl describe pod — shows events, config issues, image pull errors.
- kubectl logs — shows application startup errors.
- Verify image tag is correct. Verify ECR credentials/pull secret.
- Check if resource limits (CPU/memory) are too low for the container.

Q: Scenario: Trivy reports critical vulnerabilities. What actions do you take?

- Do not push the image to production.
- Identify which package/library has the CVE.
- Update base image to a patched version (e.g., FROM node:20-slim instead of node:18).
- Upgrade vulnerable application dependencies.
- If no fix is available: Apply Trivy suppression with documented justification and set a review date.

Q: Scenario: Pipeline works in staging but fails in production. How do you investigate?

- Compare environment variables and secrets between staging and production.
- Check IAM permissions — production may have stricter policies.
- Review Kubernetes resource quotas and namespace configurations.
- Check if production uses a different Kubernetes version or node configuration.
- Look at application logs in production for specific error messages.

Q: Scenario: GitHub Actions workflow takes 30 minutes. How do you optimize it?

- Cache dependencies: Use actions/cache for npm, Maven, pip packages.
- Parallelize jobs: Run tests and security scans in parallel jobs.
- Use smaller base images for Docker builds.
- Use multi-stage Docker builds to cache layers.
- Skip unnecessary steps: Use path filters to run only when relevant files change.
- Use self-hosted runners with better hardware for compute-intensive jobs.

13. AWS CI/CD Integration

Q: How does Jenkins authenticate to AWS?

- Best practice: Attach an IAM Role to the EC2 instance running Jenkins.
- IAM role grants permissions without storing access keys.
- Alternative: Store AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY in Jenkins Credentials.
- Use least-privilege IAM policies — only grant permissions the pipeline actually needs.

Q: How do pipelines push images to AWS ECR?

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS
--password-stdin <account_id>.dkr.ecr.us-east-1.amazonaws.com
docker push <account_id>.dkr.ecr.us-east-1.amazonaws.com/my-app:v1.0
```

- Authenticate to ECR using AWS CLI.
- Tag image with ECR registry URL.
- Push the image.

Q: How do you manage multiple AWS environments (dev, staging, prod)?

- Use separate AWS accounts or separate VPCs for each environment.

- Use different IAM roles per environment.
- In Terraform: Use workspaces or separate state files per environment.
- In pipelines: Use environment-specific variables/secrets for each stage.
- Protect production: Require manual approval before any production deployment.

14. Pipeline Design Questions

Q: Design a complete CI/CD pipeline for a Java application deployed to EKS.

- Stage 1 - Checkout: Pull code from GitHub.
- Stage 2 - Build: mvn package to compile and package the JAR.
- Stage 3 - Unit Test: mvn test to run unit tests.
- Stage 4 - SonarQube Scan: Static analysis and quality gate check.
- Stage 5 - Docker Build: Build Docker image with commit SHA as tag.
- Stage 6 - Trivy Scan: Scan Docker image for CRITICAL CVEs.
- Stage 7 - Push to ECR: Tag and push the image to AWS ECR.
- Stage 8 - Deploy to Staging: kubectl apply to staging namespace.
- Stage 9 - Integration Test: Run smoke tests against staging.
- Stage 10 - Manual Approval: Team lead approves production deployment.
- Stage 11 - Deploy to Production: kubectl apply to production namespace.
- Stage 12 - Verify: kubectl rollout status to confirm healthy deployment.
- Stage 13 - Notify: Send Slack message with deployment result.

Q: How would you implement manual approval in a Jenkins pipeline?

```
stage('Approve Production') {
  steps {
    input message: 'Deploy to Production?', ok: 'Approve'
  }
}
```

- The input step pauses the pipeline and waits for a human to click Approve or Abort.
- Only authorized users (configurable) can approve.
- Can set a timeout: if no approval within 1 hour, pipeline aborts automatically.

15. Behavioral Questions — How to Answer

Q: Tell me about a CI/CD failure you investigated.

Structure your answer using STAR: Situation, Task, Action, Result.

- Situation: Describe the context — what system, what stage of the pipeline failed.
- Task: What was your responsibility in resolving it?
- Action: Walk through your exact troubleshooting steps — logs checked, root cause found.
- Result: How was it fixed? What did you learn? What process was improved?

Q: Describe a pipeline you designed from scratch.

- Start with the business requirement: what was the application, what did deployment look like before?
- Explain the tools you chose and why: Jenkins vs GitHub Actions, Docker, ECR, EKS.
- Walk through each stage you added and the rationale.
- Share the outcome: time saved, bugs caught, deployment frequency improved.

Q: What was your biggest CI/CD improvement?

- Be specific: e.g., 'Reduced build time from 20 minutes to 8 minutes by adding dependency caching and parallelizing test and scan stages.'
- Or: 'Caught a critical SQL injection vulnerability via SonarQube before it reached production.'

• Quantify where possible. Interviewers value concrete impact over generic statements.

Prepared for DevOps / Cloud / Platform Engineer / SRE Interviews | Focus: Jenkins, GitHub Actions, AWS, Docker, Kubernetes, Terraform, SonarQube, Trivy